Guiding the Grid: An Algorithmic Optimisation to the Computer Game *Griddo*

[redacted] - VCE Units 3/4 Algorithmics (HESS) SAT Project

May 25, 2025

Scenario and Outline

In the computer game 'Griddo', players move characters around a map. The map is a two-dimensional, square grid. Each grid square has a type of terrain.

- Field squares allow for normal movement.
- Forest squares can be moved through, but only at half speed.
- Mountain squares cannot be moved through.

During the game, both field and forest squares may be temporarily modified with rain, halving normal movement speed through that square.

The location of fields, forests and mountains does not change during the game, hence, it is possible to pre-compute the shortest paths between locations on the map without the presence of rain.

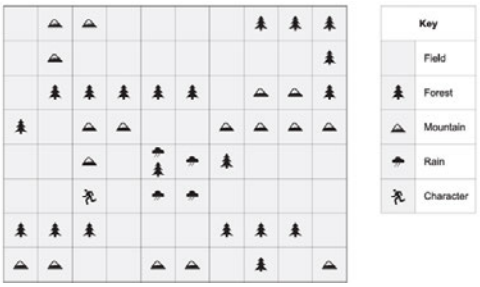


Figure 1: An example of a small map. Maps in the actual game can have millions of squares.

Characters can move up, down, left or right to adjacent squares.

During the game, players can instruct their character to move to a location. When this is done, the game needs to determine the path that the character will take to get there and begin moving the character. It needs to determine the path quickly and also find a fast path for the character to take to get to the destination.

The goal of this project is to investigate an algorithmic approach to solving this problem, using a range of different Abstract Data Type (ADT) and Algorithmic Design concepts. This will take the form of a written report in **Sections 1-4**, using pseudocode to plan out the process of the algorithm, then analysing the efficiency of the pre-existing code using time complexity, before finally implementing an advanced and improved version in the Python programming language.

Note for Marker (Neil) - Section-Criterion Correlation:

In Part One of the SAT, Section 1 addresses Criterion 1, but with incomplete signature specifications. Section 2 addresses Criterion 2, along with the rest of Criterion 1's signatures in 2.5, as well as a part of Criterion 4 throughout the design process. Section 3 addresses Criterion 3, and Section 4 addresses the remainder of Criterion 4.

1 Model and Specification

1.1 Modelling Features with Abstract Data Types

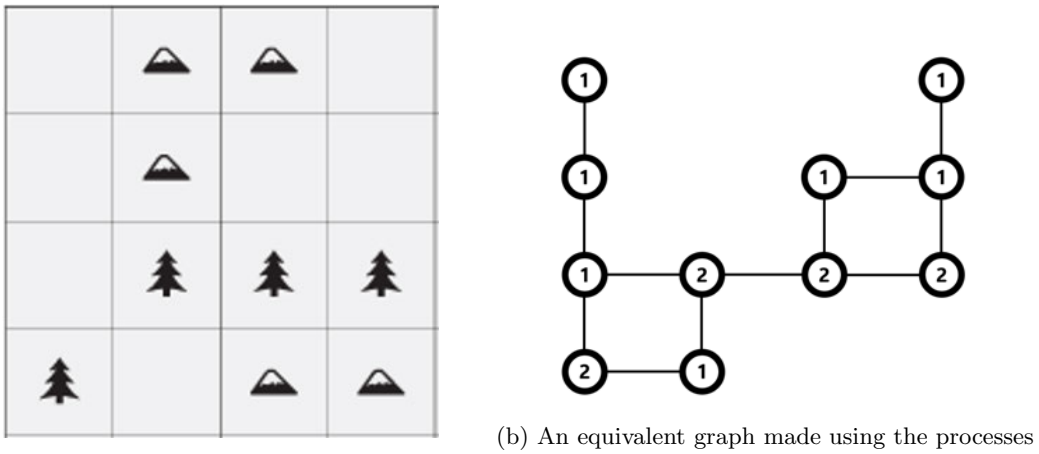
The aim of Part One of the project is to create an algorithm that will find an optimal solution to every possible path-finding task the game must run, using techniques learnt in Unit 3 of the subject. The model assumes access to the standard ADTs of lists, arrays, sets, queues, stacks, dictionaries, and graphs, as well as PDTs of integers, strings, booleans, and floats. For simplicity and suitability, the squares on the map will be referred to as tiles throughout the project.

1.1.1 Turning the Map into an ADT

In order to begin modelling the problem, it must first be considered what exactly is being done. The player must start in the centre of a tile, and traverse through multiple other tiles to reach the centre of the end tile. This process must be done efficiently both within the code (time complexity) and within the scope of the game (shortest path); time complexity will be discussed in parts two and three, but the main focus of part one remains on finding the shortest path possible.

The Abstract Data Type of a Graph, comprised of Edge ADTs and Node ADTs, will be used to model this problem, as all properties of the Griddo map can be clearly represented by one. An orderly grid of nodes each represent the corresponding tile on the map, and four edges protrude out of each node connecting it to all of its adjacent (non-diagonal) neighbours representing the direct connection for the player to traverse to adjacent tiles. These edges are undirected, as they can be traversed in either direction with equal speed. As with most standard graphs, each node will have two corresponding List ADTs to store incident edges and neighbouring nodes.

An integer node property will be stored on every vertex representing its movement cost to travel through; **1** for fields as it can be moved through at "normal" speed, **2** for forests as it takes twice as long to move through, and an indefinitely large cost for mountains as they cannot be moved through. All nodes with this very large cost, and edges connecting to these nodes, will be removed as they represent impassable mountains.



(a) An example 4x4 Griddo area

(b) An equivalent graph made using the processes stated above

Figure 2: A demonstration of how the graph is used to model the Griddo map.

Whilst not directly necessary for the model and algorithm, it is worth noting that if the nodes were to require a name for identification, they would be easily indexed with a numerical string counting left to right, then top to bottom. For example, a 4x4 map like the one in Figure 2 could be indexed and named in the string as such:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

As an extension of this, edges would also be simply identified by referencing the two nodes it connects (e.g. the edge connecting node "6" and node "10" would be named "6-10").

1.1.2 The Player

The matter of the player is an interesting conundrum. It could be easily stored as a boolean node property on tiles to indicate if the player is currently on the tile, but that lacks the depth required for this integral part of the game, and would be incredibly inefficient to attempt to access regularly as every single node would need to be searched to find its location. Hence, a new ADT must be defined.

We will call this the Player ADT. Its definition will contain a single data type, the node it is on. This allows for easy access of the starting node for path-finding, and to traverse the player along tiles with a single call. Its operations are defined in 1.2.

1.1.3 Calculating Movement Costs

For accuracy to the scope of the game, it is safe to conjecture that the player, starting in the centre of each tile, must traverse to the end of the tile, enter into the next tile, then traverse another halfway through the new tile to reach the new centre. Hence, the movement cost required to travel between tiles can be calculated as the average of the movement cost of the two nodes connecting the edge, and stored as the edge weight between nodes as a float. The resulting movement cost to traverse between nodes is listed below.

- Field ↔ Field: **1**
- Field ↔ Forest: **1.5**
- Forest ↔ Forest: **2**

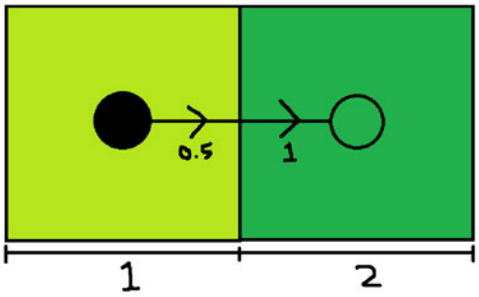


Figure 3: A demonstration of the calculation. This represents the player moving from a field to a forest.

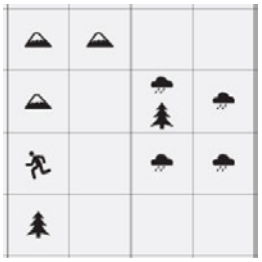
However, rain also randomly occurs on field and forest tiles throughout the game. It is not predictable, and as stated in the original scenario and project FAQ, may modify tiles each time a single tile is traversed by the player. It affects the movement cost of each node by doubling it.

To model this, whenever rain occurs and modifies a tile, the node property storing cost will be temporarily modified and doubled (i.e. **2** for fields and **4** for forests), and all of its incident edges' weights will be recalculated with a new average. When the rain ends, the cost will be halved to return it to its original value, and the edge weights will once again be recalculated. For the sake of clarity and completeness, a list of these modified movement costs is also listed below.

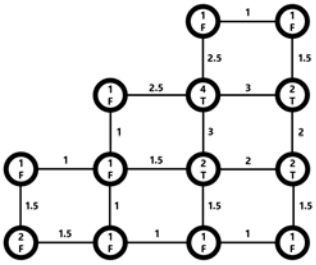
- Raining Field ↔ Field: **1.5**
- Raining Field ↔ Forest: **2**
- Raining Field ↔ Raining Field: **2**
- Field ↔ Raining Forest: **2.5**
- Forest ↔ Raining Forest: **3**
- Raining Field ↔ Raining Forest: **3**
- Raining Forest ↔ Raining Forest: **4**

Furthermore, an additional boolean node property will be stored on each node, indicating whether it is currently raining or not on the tile.

From this model, it can be clearly seen that this is simply a shortest path (lowest cost) problem on a weighted graph, with the player's position as the starting node and the tile the player selects as the ending node; bearing in mind the grid map with terrain modifiers. However, the randomness of rain and how it affects the graph adds another layer of complexity with changing path lengths over time, potentially causing the need for multiple recomputations of shortest paths to be found, and this will be addressed in Section 2.



(a) A 4x4 Griddo area. Note the rain and player.



(b) An equivalent graph made using the additional processes stated above

Figure 4: A further demonstration of how the graph is used to model the Griddo map.

1.2 Signature Specifications

The signatures required to create, use, and access the base model as intended are as follows:

Graph ADT:

- newMap: → graph *(generates unweighted undirected grid graph of the Griddo map)*
- declutterMap: graph → graph *(removes all mountains and edges connected to them)*

Node ADT:

- assignTile: node × integer → node *(assigns movement cost of tile type as a node property)*
- tileType: node → integer *(returns movement cost)*
- tileName: node → string *(returns name of node)*
- neighbours: node → list *(returns list of neighbouring nodes)*
- incident: node → list *(returns list of connected edges)*
- spawnRain: node → node × edge × edge × edge × edge *(doubles the movement cost of the node, sets the boolean to True, and recalculates all incident edge weights)*
- clearRain: node → node × edge × edge × edge × edge *(halves the movement cost of the node, sets the boolean to False, and recalculates all incident edge weights)*

Edge ADT:

- calcWeight: edge × node × node → edge *(calculates and assigns edge weight)*
- edgeWeight: edge → float *(returns weight of edge)*
- edgeName: edge → string *(returns name of edge)*
- connectedTiles: edge → node × node *(returns tiles the edge is connected to)*

Player ADT:

- position: player → node *(returns player's current position)*

This list of signatures is not fully comprehensive, as it only covers those used in the initial modelling of the problem. Further traversal and path-finding related signatures will be provided in Section 2.5, after the algorithm is designed.

2 Designing the Algorithm

2.1 Suitability of the Options Available

There are many types of algorithm design techniques that could be applied on a graph ADT.

However, Graph Traversal Algorithms such as Breadth-First Search and Depth-First Search, as well as Minimum Spanning Tree Algorithms such as Prim's Algorithm and Kruskal's Algorithm, will not be discussed as they are not applicable to the problem (breadth-first only finds the shortest path in an unweighted graph and depth-first is only useful in a maze-type problem, neither of which are the case here), and/or they do not provide the solution we are looking for (minimum spanning trees are not the end goal).

It is quite clear from the initial model that the goal of this algorithm is to find the shortest path; hence, we will make use of shortest path algorithm design techniques to solve this problem.

2.1.1 Shortest Path Algorithms

Greedy Algorithms could be implemented in the form of a variation of Dijkstra's Shortest Path Algorithm, Dynamic Programming principles might be applied through the use and manipulation of Bellman-Ford's Shortest Path Algorithm or Floyd-Warshall's All-Pairs Shortest Paths Algorithm, or heuristics could be used in a Best-First Search. The suitability of these potential options will be discussed below.

The Best-First Search algorithm immediately appears to be a decent option, as it uses a heuristic to quickly find the best, shortest path. However, the issue arises of actually determining a good heuristic for an optimal solution, especially with the inconsistency of rain occurring randomly. Failure to find a perfect or near-perfect heuristic will result in the algorithm not being able to solve correctly for the shortest path, so it is not be an ideal choice in this situation.

Floyd-Warshall's All-Pairs Shortest Path Algorithm is another possibility, as it directly finds the shortest path between any pair of two nodes in a weighted graph. As stated in the original scenario, it is possible to pre-compute the shortest paths between locations on the map without the presence of rain, so it is viable to use Floyd-Warshall's Algorithm to achieve this. This alone will not be enough however, so another algorithm needs to be considered for when rain interrupts the movement process.

We are left with Bellman-Ford's and Dijkstra's Shortest Path Algorithms. Both of these achieve the same goal of finding the shortest path between two nodes in a weighted graph, and are completely suitable for the remainder of this problem as they will both work as intended to determine the best result.

The difference between the two is that Bellman-Ford is suited to graphs with negative-weighted edges, and finds the shortest path from the start node to all other nodes in graph, whereas Dijkstra's Algorithm is more efficient, and can be modified to only find the path from the start node to one specific end node.

Time Complexity is not an important consideration for now, but as the Griddo map will never contain any negatively weighted edges, and this algorithm is only needed to find the path between two specific points, we will select Dijkstra's as a more optimal and efficient choice to supplement Floyd-Warshall's where necessary.

2.2 The Optimal Base Algorithms

2.2.1 Dijkstra's and Floyd-Warshall's

It has been decided that a combination of two algorithms will be used to optimally solve this problem; Dijkstra's Algorithm, and Floyd-Warshall's Shortest Path Algorithm. However, they cannot simply be directly applied to the graph that has been modelled, as there is much more depth and many more intricate layers to consider. Before making any changes to fit the model, the original algorithms will first be expressed.

Dijkstra's Algorithm: Dijkstra's is a well-known algorithm used to find the shortest paths from one node to every other node on a weighted graph that do not contain negative-weighted edges, and returns the lengths as an array. However, as we only require one specific path from the start node to target end node, we will modify the algorithm such that it does not unnecessarily compute other shortest paths.

It is important to note that the minimum priority queue version of Dijkstra's Algorithm is used here as it is more efficient and much easier to process in general.

1. Initialise two ADTs of all nodes; a minimum priority queue of all currently unsearched nodes (starts with all nodes in it), and an array of integers of the current shortest distances to every node (set starting node to 0, the rest to an indefinitely large number). *(minimum priority queue is used so the node with the smallest distance is easily accessible)*

2. Assign priority indexes to all unsearched nodes, equivalent to their distance from the starting node.

3. Select the node with the smallest priority index as the current node, and remove it from the unsearched nodes queue (marking it as searched).

4. For each unsearched neighbour of the current node, if the distance of the current node + the distance from the current node to said neighbour is shorter than its current distance, update the array of shortest distances with the new value.

5. Repeat from **step 2** until the desired end node is searched, and return the length of the shortest path from the starting node to that end node.

Floyd-Warshall's Algorithm: Floyd-Warshall, specifically the All-Pairs Shortest Path version, finds the shortest path between every possible pair of nodes in a weighted graph without negative-weighted cycles, and returns the lengths as a matrix.

1. Initialise a square matrix (two-dimensional array) with length equal to the number of nodes; this will store the shortest paths between every pair of nodes (row = starting node, column = ending node). Start all values at an indefinitely large number, except the lead diagonal (where the start and end nodes are the same) which remains at 0.

2. For every possible pair of nodes, check if they are neighbours, and if so, update the lengths in the matrix with the weight of the edge between them.

3. Iterate through every node k, and for every pair of nodes [i,j], check if the distance of [i,k] + the distance of [k,j] is less than the current distance of [i,j]. If so, update the shortest path length of [i,j] to the distance of [i,k] + the distance of [k,j].

4. Once all nodes have been iterated through, return the final matrix of shortest paths.

2.2.2 Returning the Shortest Paths

In order to apply this correctly to Griddo, a slight modification needs to be made; it should also store the shortest path itself, so that it can be returned and used within the game to move the player.

For Dijkstra's Algorithm, this can be done by additionally initialising an array of lists of nodes, one list for each node to record its current shortest path, with the starting node's list just containing itself at the start of the algorithm.

Then, when a new shortest path is found to a node, the path will be updated by setting its list to the list of the previous node, and appending the end node to the end of the list. For example, if a new shortest path was found from A to D using the edge B-D, and node B had the shortest path [A, F, B], D's shortest path would be set as [A, F, B, D].

For Floyd-Warshall's Algorithm, a matrix of lists of nodes would be used correlating to the distances matrix, with adjacent nodes i, j being initialised as [i, j] (or [j, i] if going the other way).

Then, when a new shortest distance from i to j is updated through k, set the shortest path list to that of [i,k] with [k,j] appended on the end without [k]. For example, if a new shortest path was found from A to D using node C, and the shortest path lists of A-C and C-D were [A, B, C] and [C, E, D] respectively, the shortest path list of A-D would be [A, B, C, E, D]. This concatenates the two paths together without causing a repeated node in the list.

This list and matrix of shortest paths would be updated alongside the shortest path lengths in their respective algorithms, and returned instead of these lengths at the end.

There may exist multiple shortest paths with equal weights, but there will not be any significant difference between the two in terms of overall pathfinding length in the full algorithm, if any, so the one obtained first by the algorithms is fine to use.

2.2.3 Considering Unreachable Tiles

One additional thing to consider is the possibility that there is no path between the two nodes.

This is incredibly unlikely to happen, but if an area is surrounded by mountains, there should be measures in place to ensure the game does not try to traverse to the route and instead immediately cancels the operation.

This can be achieved by ensuring that Floyd-Warshall's Algorithm will return an empty path list for tiles where transitive closure does not exist. This means that the iteration through intermediate nodes (k) would be the inner loop rather than the outer loop, so that a check can be run at the end of each node pair [i,j]'s searches to set the path list to empty if needed.

When initialising the traversal algorithm, check if the precomputed list returns as empty. If so, that means there is no path from the player to its selected tile and the algorithm will hence automatically cancel. This could probably be reflected in game with some sort of UI popup, but how the frontend handles it is not within the scope of the project.

2.3 Integration into Griddo

2.3.1 Considering Rain

With the base algorithms complete, the larger overall algorithm to run the full Griddo map correctly can be designed. The key additional consideration here is the presence of rain. Rain has the chance to spawn on any tile immediately after a single tile is traversed.

If one of the tiles within the precomputed shortest path becomes rained on, it is likely that it will no longer be the shortest path, and the algorithm must hence be swiftly recomputed. This is what Dijkstra's Algorithm achieves; when the precomputed shortest path no longer becomes viable, it will find a new one.

However, it is possible that the original shortest path clears up and is once again viable to use, making it a faster and thus preferred option than Dijkstra's path. This can be solved by checking the precomputed shortest path every time a new tile is traversed, and if there is no rain, turning the path back to that of the precomputed one.

Additionally, it is not enough to run Dijkstra's solely once, as it is possible that rain appears on the new path while traversing it. In order to check this, the initial state of rain on each tile in Dijkstra's shortest path will be stored, and if new rain appears on a previously dry tile in the path, potentially rendering it undesirable, Dijkstra's will be run again to find the new shortest path.

2.3.2 The Algorithm

With all of this in mind, it is clear that a decrease and conquer approach will be the most effective method of solving this problem using a combination of Dijkstra's Algorithm and Floyd-Warshall's Algorithm.

Floyd-Warshall's Algorithm will be used to provide an effective pre-computation of the map so that paths can be instantly retrieved from the matrix when rain does not interfere. However, when rain does appear, Dijkstra's Algorithm can be run to greedily find the new optimal path.

Doing this process once per tile movement essentially decreases the size of the problem at every iteration, getting closer and closer to the end tile with each step until it is reached. This ensures that no matter what occurs on the map, the absolute optimal shortest path is always the one being used at any moment.

Finally, the general idea of this overall process is as follows:

1. After the Griddo map is turned into a Graph, pre-compute all shortest paths between every possible pair of nodes using Floyd-Warshall's Algorithm.
2. Create a path list to store the order of nodes in which the player must traverse to reach its destination.
3. When the player selects a tile to travel to in the game, retrieve the pre-computed shortest path between the starting node (player) and ending node (selected tile). If the path is empty, there is no path, so exit the algorithm. If there is a path, check if there is rain along it. If there is no rain, use the pre-computed shortest path as the path list. Otherwise, run Dijkstra's Algorithm to replace it with a currently more optimal path.
4. Create a list of booleans the same length as the path list, storing booleans of whether the corresponding node in the path list is raining or not. Populate this correctly. If the precomputed path is being used, these values must all be false.
5. Pop the first node from the path list and the rain boolean list, traversing one step along the path.
6. Check if the first node in the path list (the next one to traverse) is currently raining, and if the rain boolean list has it marked as not raining. If so, rain has spawned on the path, so run Dijkstra's Algorithm again and replace the two lists. Else, continue using the current path.
7. Check if the pre-computed shortest path from the current tile to the desired end tile has rain on it. If it does not, it is the optimal shortest path, so replace the path list and boolean list with the pre-computed one. Else, continue using the current path.
8. Repeat from step 4 until the path list only contains the end node (i.e. the desired end node is reached).

2.4 An Example

The following series of diagrams demonstrates an execution of the algorithm on a 4x4 Griddo map, highlighting different possible scenarios in each iteration. The player starts on node 1, and has selected to move to node 16. On the map, the grey dotted line represents the pre-computed shortest path, and the black line represents the current path (where it is not the precomputed one). On the graph, the grey box represents the current node, and the green box represents the end node. Each image represents an iteration starting from step 4, and steps 1-3 have already been initialised before Figure 5.

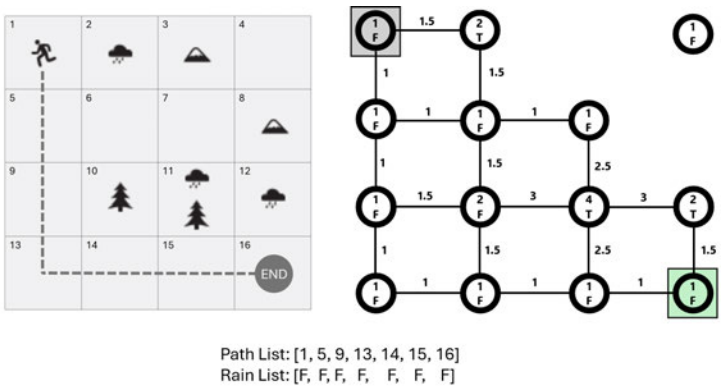


Figure 5: Precomputed path selected as it is optimal, will traverse to node 5.

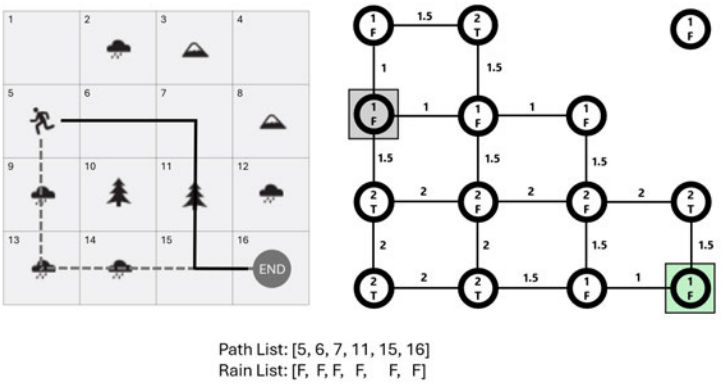


Figure 6: Rain appeared on precomputed path, so Dijkstra's was run to find new path.

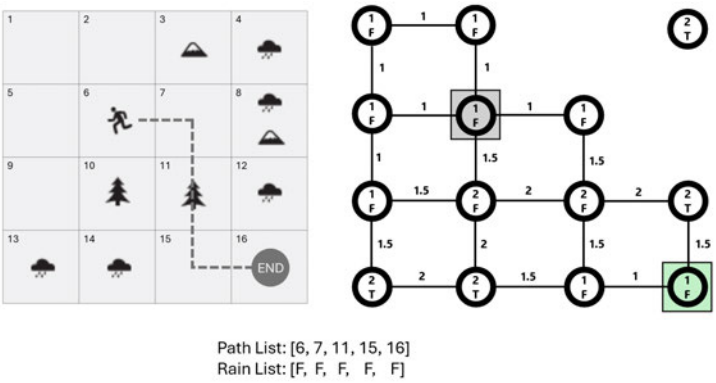


Figure 7: No new interrupting rain appeared, so continue going down same path. However, the precomputed shortest path has no rain, so replace the current path list with the precomputed one. Note that this actually doesn't change anything here as it's the same path.

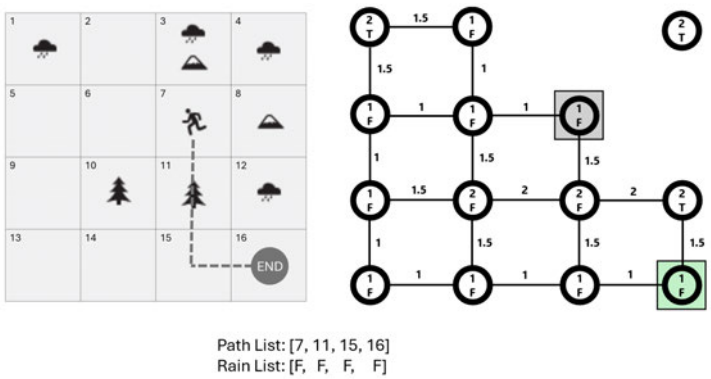


Figure 8: No new interrrupting rain appeared, so continue going down same path.

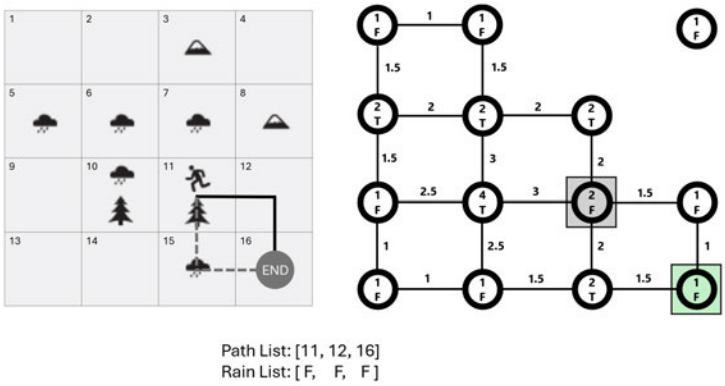


Figure 9: Path has been interrupted by rain, Dijkstra's was run to recompute new path.

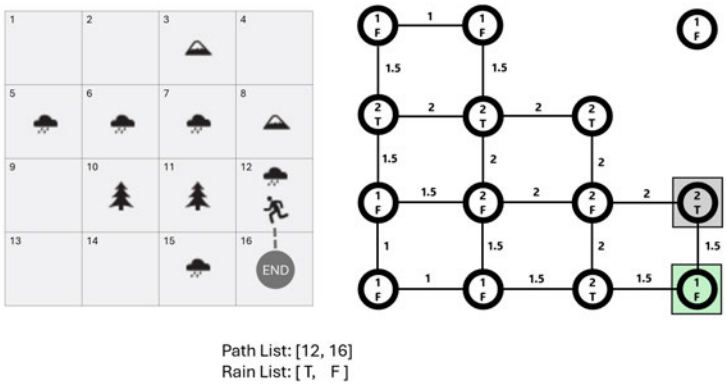


Figure 10: Rain has spawned on the character, but that does not affect the path so keep going.

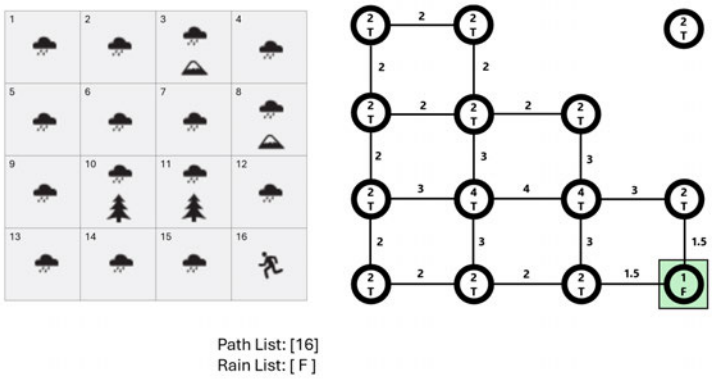


Figure 11: The player has arrived at the end node, and the algorithm is complete.
Looks like they narrowly dodged a huge rainstorm too!

It is worth mentioning that if the player had picked node 4 to traverse to, the pre-computed path would have returned with nothing and the algorithm would immediately end, as it is inaccessible.

In section 3, this process will be formalised and fully laid out in pseudocode.

2.5 Further Signature Specifications

Throughout the design of this algorithm, many new abstract data types and processes were used to operate it efficiently. The following is a list of signatures that are required to compute and run this algorithm correctly.

Graph ADT:

• precomputeMap: graph → two-dimensional array *(runs Floyd-Warshall's Algorithm on the initial rainless map to generate shortest paths)*

• selectTile: graph → node *(represents the player selecting a tile to move to)*

• generatePathList: node × node → list *(creates a list to store the path the player must traverse, initialising it with the precomputed path)*

• generateRainList: graph × list → list *(creates a corresponding boolean list to the path list to store rain status)*

• checkRain: graph × list × list → boolean *(checks if rain has generated on the next step in the path where there was not before)*

• recomputePath: node × node → list × list *(runs Dijkstra's Algorithm on the current map and replaces the path and rain lists)*

Player ADT:

• traversePlayer: list × player → list × player *(moves the player one step along the path and removes that tile from the list)*

3 Implementing the Algorithm in Pseudocode

The following pseudocode assumes that the graph model has already been created accurately according to the methods expressed in Criterion 1, including all nodes, node properties, edges, and edge weights; this includes the asynchronous update of rain on all tiles such that the node property isRaining and incident edge weights will always return the correct value at that specific game state.

Note that "indefinitely large number" is used in place of an arbitrary large number when initialising, as infinity cannot be used. "move player" refers to the Player ADT, where it will traverse along the edge that connects the current node with the next node.

FloydWarshall(), Dijkstra(), and PrecomputedOptimal() are helper functions used in the algorithm to precompute paths, recompute a path, and check for rain on a path respectively. While in code, the precomputation of shortest paths occurs every time GriddoPath() is run, in an actual game scenario the precomputed list would be initialised at the very start of the game and stored as a global variable, so it could simply be accessed anytime and this would not be required. To use the algorithm, run GriddoPath() once, and enter the Griddo map, player's current node, and desired end node as the arguments.

```
Define Dijkstra(Graph G = {V, E, w}, start, end)
    // Initialises Dijkstra's Shortest Path Algorithm with the graph and start/end nodes
    V = list of nodes in graph G
    E = list of edges in graph G
    w = weight function // list of movement costs to move between nodes
    start = player's starting tile
    end = desired end tile

    create new empty length |V| array distances // contains all lengths of shortest paths
    create new empty length |V| array pathOrder // contains all shortest paths
    create new empty minPriorityQueue unsearchedNodes // contains nodes to be searched

    for each node N in V: // Initialises all distances and shortest paths
        distances[N] := indefinitely large number
        pathOrder[N] := []
        append N to unsearchedNodes
    distances[start] := 0 // Starting node is at 0
    pathorder[start] := [start]

    while(unsearchedNodes contains end): // Iterates until the end node is fully searched
        for each node N in unsearchedNodes: // updates priority index to dist. from start
            priority index P of N = distances[N]

        currentNode := node in unsearchedNodes with lowest P // selects next node
        remove currentNode from unsearchedNodes

        for each neighbour N of currentNode: // investigate currentNode's neighbours
            if(unsearchedNodes contains N): // ensures neighbours have not been visited
                edgeWeight := w(currentNode, N)
                if(distances[currentNode] + edgeWeight < distances[N]):
                    // A shorter shortest path has been found; update stored values
                    distances[N] := distances[currentNode] + edgeWeight
                    pathorder[N] := pathorder[currentNode]
                    append N to the end of pathorder[N]

    return pathorder[end] // path list of desired shortest path
End Dijkstra

Define FloydWarshall(Graph G = {V, E, w})
    // Initialises Floyd-Warshall's All-Pairs Shortest Path Algorithm
    V = list of nodes in graph G
    E = list of edges in graph G
    w = weight function // list of movement costs to move between nodes

    create new |V| x |V| array distance // contains all lengths of shortest paths
    create new |V| x |V| array pathOrder // contains all shortest paths

    for each node vert1 in V: // Initialise shortest path lists/lengths
        for each node vert2 in V:
            if(vert1 != vert2):
                distance[vert1][vert2] := indefinitely large number
                pathOrder[vert1][vert2] := []
            else: // lead diagonal where start and end are the same node
                distance[vert1][vert2] := 0
                pathOrder[vert1][vert2] := [vert1]

    for each edge (i,j) in E: // Sets distances between adjacent tiles
        distance[i][j] := w(i,j)
        pathOrder[i][j] = [i, j]
        distance[j][i] := w(i,j)
        pathOrder[j][i] = [j, i]

    for each node i in V: // Starting node
        for each node j in V: // Ending Node
            for each node k in V: // Intermediate node to go through
                if(distance[i][j] > distance[i][k] + distance[k][j]):
                    // A faster route has been found
                    distance[i][j] := distance[i][k] + distance[k][j] // Updates distance
                    pathOrder[i][j] := pathOrder[i][k]
                    remove last node from pathOrder [i][j]
                    append pathOrder[j][k] to pathOrder[i][j] // Sets path order

            if(distance[i][j] = indefinitely large number): // No path
                pathOrder[i][j] := []
    return pathOrder // returns full matrix of precomputed paths
End FloydWarshall
```

```
Define PrecomputedOptimal(Graph G = {V, E, w}, path)
    // Checks if a pre-computed path is clear of rain and hence optimal
    V = list of tiles (nodes) in the griddo map // each node has bool property isRaining
    E = list of edges in the griddo map
    w =  weight function // list of movement costs to move between tiles in the map
    path = list of nodes which represents a traversable path on the map

    optimality := true // whether the pre-computed path is guaranteed to be optimal

    for each node N in path:
        // checks if it is raining on tile
        if(N.isRaining = true):
            optimality := false // rain is on the tile and path may hence be suboptimal
    return optimality
End PrecomputedOptimal


Algorithm GriddoPath(Graph GMap = {V, E, w}, startNode, endNode)
    // This is the main algorithm that will be run to complete a full path traversal.
    V = list of tiles (nodes) in the griddo map // each node has bool property isRaining
    E = list of edges in the griddo map
    w =  weight function // list of movement costs to move between tiles in the map
    startNode = tile where the player starts
    endNode = tile that the player has selected to move to

    precomputedPaths := FloydWarshall(GMap) // precomputed shortest paths between every
        pair of tiles
    currentNode := startNode // stores the tile the player is currently on
    currentPath := [] // stores the path the player is traversing

    if(precomputedPaths[currentNode][endNode] = []):
        // there is no path from the start to the end node
        return "There is no path from the current node to the selected end node."

    if(PrecomputedOptimal(GMap, precomputedPaths[currentNode][endNode]) := true):
        // pre-computed path is optimal
        currentPath := precomputedPaths[currentNode][endNode]
    else:
        // path needs to be recalculated
        currentPath := Dijkstra(GMap, currentNode, endNode)

    create new empty list rainStatus // stores initial rain state of currentPath
    for each node N in currentPath: // fills the rain state list
        append N.isRaining to the end of rainStatus

    while(currentPath != [endNode]): // keeps running algorithm until end is reached
        // move player one step along path
        pop the first node from currentPath
        pop the first node from rainStatus
        move player from currentNode to currentPath[0]
        currentNode := currentPath[0]

        // check if the current path has been interrupted by rain
        if(isRaining of currentPath[0] = true AND rainStatus[0] = false):
            // rain has spawned on the path and it needs to be recomputed
            currentPath := Dijkstra(GriddoMap, currentNode, endNode)

            rainStatus := [] // resets rain state list
            for each node N in currentPath: // repopulates the rain state list
                append N.isRaining to the end of rainStatus

        // check if the pre-computed shortest path is once again optimal
        if(PrecomputedOptimal(GMap, precomputedPaths[currentNode][endNode]) := true):
            // pre-computed path is optimal
            currentPath := precomputedPaths[currentNode][endNode]

            rainStatus := [] // resets rain state list
            for each node N in currentPath: // repopulates the rain state list
                append false to the end of rainStatus

    return // No output needed as the player is moved through the algorithm
End Algorithm
```

4 Evaluation of the Model and Algorithm

4.1 The Model

The model created in this project is well-suited to the real life scenario. It functionally converts the Griddo map into a graph ADT; there is no other ADT that could sensibly work well in this scenario, as the Griddo map maps perfectly onto the elements of a Graph. It retains all key information relating to the map such as tile type (stored as its movement cost), position of each tile and its neighbours, distances between nodes, and the effects of rain within ADT properties and suitable ADTs (lists, arrays, 2D arrays, etc.), allowing for quick and direct accessibility when needed. Furthermore, it is easily and indefinitely scalable in every direction, allowing for a large variety of map sizes.

At the same time, it creates a useful abstraction by removing irrelevant redundancies such as edges and nodes for the inaccessible mountains, ensuring that only the necessary details are stored and used for the next step in the algorithm. This is important, as otherwise a lot of additional time would be wasted in the algorithm computing nodes and edges that cannot even be reached or traversed.

Although the process of finding player movement costs for edge weights may appear convoluted, and the resulting potential non-integer values complex, it is a necessary move to retain the purpose of terrain. Picking simply either the maximum or minimum node's movement cost entirely neglects the respectively smaller or larger terrain's value, and so does picking only the starting or ending node. Doing so might result in the algorithm being unable to tell the difference between a field and forest when pathfinding in some circumstances, which is definitely not intended.

Hence, the average-cost calculation is the most accurate possible version of finding the edge weights, and the decimal values is mostly a non-issue as a computer would have no qualms handling more sophisticated values like these.

4.2 The Algorithm

There are many possible ways a shortest path problem like this could be solved. As established in 2.1.1, I discarded best-first search as finding an accurate heuristic for this problem would be an unnecessarily complicated process that does not even guarantee correctness, and Bellman-Ford was rejected as there are no negative edges in this graph model, and it is only able to find *every* shortest path from a point instead of a singular one. A brute-force approach was also obviously discarded as it is simply inefficient and may cause significant unnecessary movement and iterations.

Floyd-Warshall and Dijkstra's Algorithms were selected as they served their required purpose simply and well. Floyd-Warshall is our only available algorithm that can effectively precompute every path correctly, and Dijkstra's is a reliable shortest-path algorithm for positive edge weights, such as in this graph, that also has the unique capability to terminate when the desired end node is reached, saving many redundant iterations. These two algorithms' processes and results align well to the problem.

The decrease-and-conquer design approach of this algorithm is an effective method of solving the problem. Ultimately, the goal of the problem is to get from one point to another on the map in a constantly changing environment, so the ideal approach would be to continuously get closer to the end tile and counteract the dynamic nature of rain. This is what decrease-and-conquer does.

The algorithm designed in this project is overall well-suited to the real problem. Although the process itself is not trivial, its steps logically build on each other and solve the problem in such a way that no redundant calculations or path recomputations are made, only necessary ones to move the solution forward. Edge cases and worst-case scenarios such as non-existent paths are handled properly to avoid errors.

The very nature of unpredictable rain prevents any form of forward planning. Hence, the algorithm uses aspects of greedy design to ensure that the current optimal shortest path is always continuously in use, no matter what happens on the map, mostly getting the player closer to the end with each iteration. This is the only feasible computational method that can provide an accurate answer; even if it may not give the true optimum solution in every scenario, it is simply impossible to construct an algorithm that is capable of predicting the future to guarantee the optimum solution, so an algorithm such as this that can reliably find a reasonably optimal solution in all scenarios is the best resolution to the Griddo problem.